

AWS 서버리스 자가복구 & 관측성 플랫폼

장애를 스스로 감지해 마지막 정상 버전으로 되돌리고, 어떤 경우에도 예산을 넘지 않도록 설계한 서버리스 API

GitHub github.com/dodongwon33/aws-serverless-selfhealing · Terraform · GitHub Actions · Python · 서울 리전

2단

자동 롤백
(카나리 + 런타임)

≈ \$0.6

최악 월 비용
(무료 티어 미적용)

31건

초안 설계 결함
발견 · 수정

0개

장기 Access Key
(OIDC)

59개

Terraform 리소스
(모듈 9개)

문제 정의

개인이 운영하는 서비스는 장애를 24시간 지켜볼 사람이 없고, 실수 한 번(트래픽 폭주, 잘못된 배포)이 곧바로 청구서 폭탄이나 장시간 장애로 이어진다. 이 프로젝트의 목표는 세 가지다.

- **보이게 한다** — 로그·메트릭·트레이스로 "무엇이, 어느 버전에서, 왜" 실패했는지 추적
- **스스로 되돌린다** — 사람의 개입 없이 수 분 안에 마지막 정상 버전으로 복구하고, 같은 결함이 다시 배포되지 않게 막음
- **예산을 넘지 않는다** — "무료 티어라 괜찮다"가 아니라, 공격·실수 상황에서도 비용 상한이 구조적으로 보장되게 함

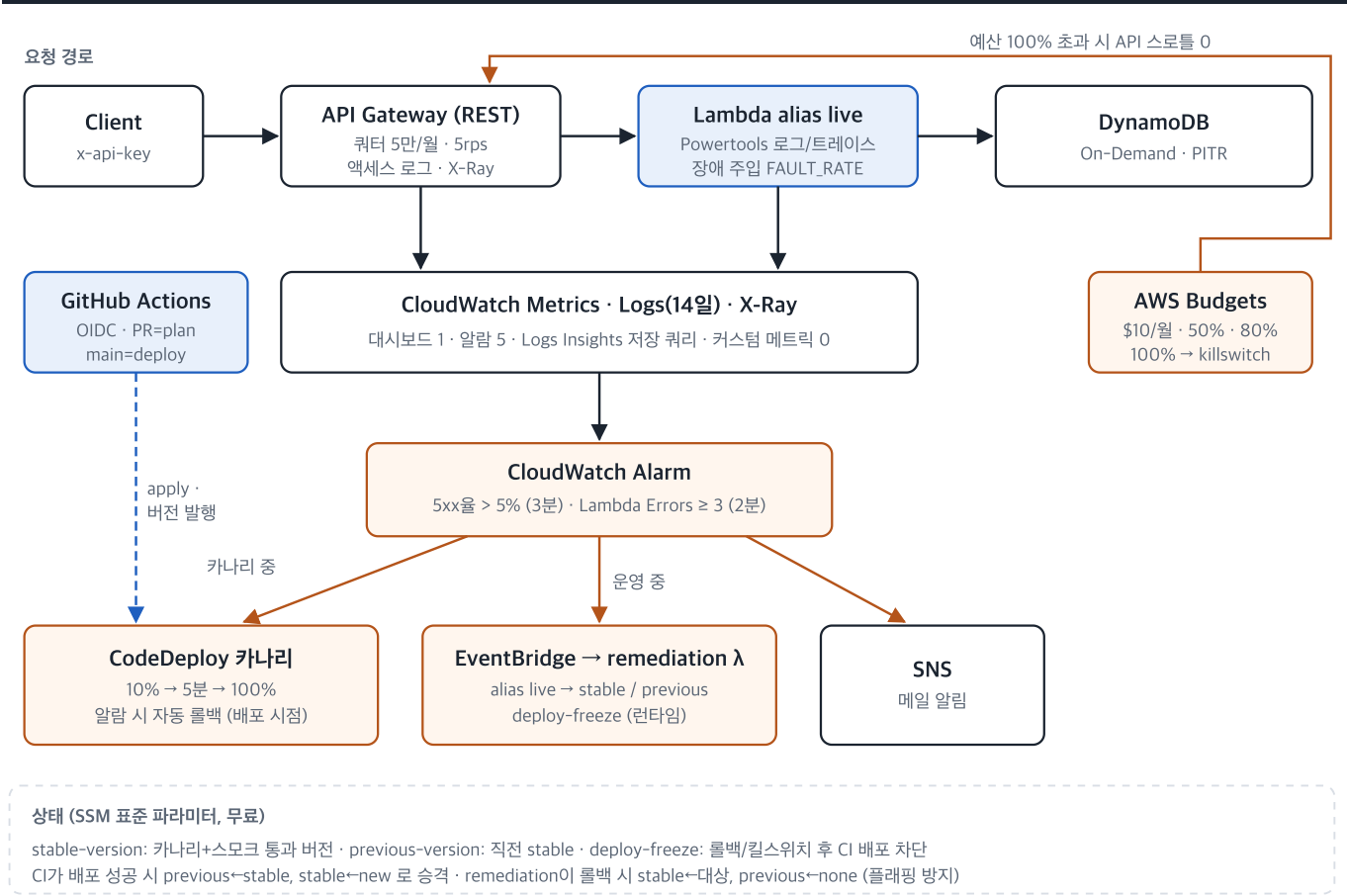
핵심 설계 결정

결정	이유
REST API (HTTP API 아님)	X-Ray 트레이싱과 사용량 계획(월 쿼터)은 REST 전용. 쿼터 상한 기준 요금 차이는 월 \$0.1 미만
API → Lambda alias 연결	\$LATEST 를 호출하면 롤백이 트래픽에 반영되지 않음. alias 하나로 카나리·롤백을 모두 제어
CodeDeploy 카나리 + 직접 구현 remediation	배포 시점은 관리형 서비스에 맡기고, 배포 이후(카나리를 우회한 핫픽스, 운영 중 드러난 결함)는 직접 구현해 둘 다 커버
코드 소유권 분리	Terraform은 함수 설정만, CI는 코드·버전만. 같은 리소스를 두 도구가 덮어쓰는 드리프트 제거
쿼터 = 비용 상한	스로틀은 속도 제한일 뿐 총량 제한이 아님(10rps면 월 2,592만 건 ≈ \$90). 월 5만 건 쿼터로 API 비용 상한 ≈ \$0.2
권한 경계(Permissions Boundary)	IaC 배포 역할이 "관리자 역할을 만들어 탈취"하는 권한 상승 경로를 구조적으로 차단

기술 스택

API Gateway · Lambda · DynamoDB · CloudWatch(Logs/Metrics/Alarms/Dashboard/Logs Insights) · X-Ray · EventBridge · SNS · CodeDeploy · SSM Parameter Store · AWS Budgets · IAM OIDC · Terraform 1.10+ (AWS provider 6.x) · Python 3.12 · Powertools for AWS Lambda · GitHub Actions · pytest / moto

아키텍처



컴포넌트

구성	설정	비고
API Gateway	GET /health · POST /items · GET /items/{id}, API 키 필수, 스로틀 5rps/버스트 10	실행 로그·상세 메트릭 OFF (비용)
Lambda (API)	Python 3.12, 256MB, Powertools 레이어(버전 고정), X-Ray Active, 본문 10KB 제한	설정=Terraform, 코드=CI
Lambda (보조)	remediation(알람→롤백), killswitch(예산→차단), 128MB	권한 경계 적용
DynamoDB	items (PK id), On-Demand, PITR, AWS 소유 키 암호화	월 1원 미만
Terraform	bootstrap(state:OIDC·역할) + dev 환경, 모듈 9개, S3 네이티브 락	DynamoDB 락 테이블 불필요

자가복구 — 2단 자동 롤백

롤백 기회는 두 번이다. 배포하는 순간에는 CodeDeploy가 트래픽 10%로 새 버전을 5분간 관찰하고, 배포 이후에는 알람이 EventBridge를 거쳐 직접 구현한 remediation 함수를 깨운다.

런타임 remediation 의사결정

1. 필터 ALARM 상태가 아니면 무시	2. 위임 카나리 진행 중 / 최근 20분 내 실패 → CodeDeploy 소관	3. 중복 방지 이미 동결 + live=stable → 추가 조치 없이 알림	4. 대상 결정 stable / previous / 수동	5. 실행 alias 갱신 → 상태 갱신 → 배포 동결 → 메일
----------------------------------	--	--	---	---

상황	버전 상태	처리
카나리를 우회한 핫픽스가 장애	live=7, stable=5	v5로 롤백
검증을 통과한 버전이 운영 중 장애	live=stable=5, previous=4	v4로 롤백, previous 비움
되돌아갈 버전 없음 (최초 배포)	live=stable, previous=none	롤백 없이 동결 + "수동 조치 필요" 메일
두 알람(5xx율·Errors) 동시 발생	첫 실행이 이미 롤백·동결	두 번째 실행은 <code>already_frozen</code>

설계하며 막은 실패 모드

연쇄 롤백 CodeDeploy가 먼저 롤백을 끝낸 뒤 알람 이벤트가 늦게 도착하면 "진행 중 배포 없음"으로 보여, 정상 버전을 한 단계 더 되돌릴 수 있다 → 최근 20분 내 실패·중단 배포도 CodeDeploy 소관으로 판단.	플래핑 롤백 후 같은 알람이 다시 올리면 나쁜 버전으로 되돌아갈 수 있다 → 롤백된 버전을 새 stable로, previous는 비워 되돌아갈 경로를 끊음.
재배포 사고 자동 롤백 직후 다음 push가 같은 결함을 다시 배포 → <code>deploy-freeze</code> 로 CI 차단, 원인 수정 후 수동 해제.	저트래픽 카나리 개인 서비스는 평소 요청이 0건이라 카나리 5분 동안 알람이 판단할 데이터가 없다 → 카나리 동안 CI가 합성 트래픽(2rps × 7분) 발생, 알람은 분당 20건 미만이면 0%로 처리해 오탐 방지.

복구를 증명하는 장애 시연

Lambda 버전은 환경변수까지 스냅샷한다는 점을 이용했다. `scripts/chaos_bad_release.sh` 는 `FAULT_RATE=0.5` 인 버전을 발행해 live에 직접 올리고(카나리 우회), 3rps 부하를 건다. 예상 흐름은 다음과 같다.

0분 장애 버전 live 반영, 응답의 약 50%가 502	~3분 5xx율 알람 ALARM	~3.5분 EventBridge → remediation → alias를 stable로	~4분 200 응답 복귀, 메일 수신, 배포 동결
--	-----------------------------	--	---------------------------------------

구현 상태: 코드와 IaC는 완성됐고 단위 테스트·정적 검증·실제 계정 대상 `terraform plan` 까지 통과했다. 실제 배포와 장애 시연 결과(대시보드 캡처, 복구 시간 측정)는 배포 후 이 섹션에 추가할 예정이다.

CI/CD

워크플로	트리거	단계
ci.yml	Pull Request	ruff · pytest · terraform fmt/validate → 읽기 전용 역할로 plan → PR 코멘트
deploy.yml	main push · 수동	테스트 → 배포 동결 확인 → terraform apply → 버전 발행 → CodeDeploy 카나리(합성 트래픽 포함) → 스모크(health + 쓰기/읽기 왕복) → stable 승격

AWS 연결 전(저장소 변수 미등록)에는 배포 job이 자동으로 건너뛰어지고 테스트만 실행된다. 동시 배포는 concurrency로 직렬화했다.

보안

장기 Access Key 0개

GitHub OIDC로 실행마다 임시 자격증명을 발급한다. 신뢰 정책의 sub 클레임으로 역할을 분리했다.

- repo:…:ref:refs/heads/main → 배포 역할
- repo:…:pull_request → 읽기 전용 plan 역할

권한 경계로 권한 상승 차단

Terraform 배포 역할은 IAM 역할을 만들 수 있어야 하므로, 그대로 두면 관리자 역할을 만들어 탈취할 수 있다. 배포 역할은 iam:PermissionsBoundary 조건을 만족하는 역할만 /selfheal/ path에 생성할 수 있고, 경계를 제거하거나 bootstrap 역할을 수정할 수 없다.

리소스 스코프

Lambda·DynamoDB·SNS·SSM·CloudWatch·CodeDeploy 권한을 selfheal-* ARN으로 한정했다. API Gateway는 ARN이 랜덤 ID라 리전 단위로 한정했다(알려진 한계).

비밀 정보 관리

알림 메일은 GitHub Secret → TF_VAR로 주입하고 sensitive로 표시해 PR plan 코멘트에 노출되지 않게 했다. API 키는 CI에서 마스킹한다. state 버킷은 버전관리·SSE-S3·퍼블릭 차단·TLS 강제를 적용했다.

SNS를 암호화하지 않은 것은 의도한 선택이다. CloudWatch 알람은 AWS 관리형 키(aws/sns)로 암호화된 토픽에 발행할 수 없고, 고객 관리형 KMS 키는 월 \$1로 예산 대비 과하다. 토픽에는 알람 메타데이터만 흐른다.

비용 설계 — "무료 티어"가 아니라 "상한"

2025년 7월 프리티어 개편 이후 기존 계정은 12개월 무료 혜택(API Gateway 등)이 만료됐을 수 있다. 그래서 **무료 티어가 전혀 없다고 가정하고**, 쿼터 월 5만 건을 전부 소진하는 최악의 경우로 계산했다.

항목	산정 근거	월
API Gateway REST	5만 건 × 약 \$4/백만	\$0.20
Lambda	요청 \$0.2/백만 + 0.25GB × 0.2초 × 5만	\$0.05
DynamoDB On-Demand	쓰기 약 1만 건, 저장 수 MB	\$0.02
CloudWatch Logs	수집 약 0.1GB × \$0.76	\$0.08
X-Ray	기본 샘플링, 최대 5만 트레이스 × \$5/백만	\$0.25
알람 5 · 대시보드 1 · 저장 쿼리	상시 무료 한도(알람 10 · 대시보드 3) 이내	\$0
S3 · SSM · EventBridge · SNS · CodeDeploy · Budgets	무료 또는 극소. Budgets는 Budget Action 미사용이라 무료	~\$0.01
합계 (최악)		≈ \$0.6 (약 900원)

다층 예산 방어 — AWS에는 하드 지출 상한이 없다

① 쿼터 사용량 계획 월 5만 건 = 요청 수 자체의 상한 (실질적 비용 상한)	② 조기 경보 Budgets \$10(계정 전체): 실제 50% · 예측 80% 메일	③ 킬스위치 실제 100% → SNS → Lambda → API 스로틀 0 + 배포 동결
---	---	--

킬스위치는 Budget Action(유료 가능)이 아니라 SNS 알림 → Lambda로 구현해 추가 비용이 없다. Budgets 데이터는 수 시간 지연 되므로 킬스위치는 마지막 안전망이고, 실질적인 상한은 ①이다.

의도적으로 쓰지 않은 것

제외	이유
NAT Gateway · VPC 내 Lambda	NAT 하나에 월 약 \$43. 이 워크로드는 VPC가 필요 없음
API GW 메서드 상세 메트릭	유료 커스텀 메트릭으로 과금. 스테이지 기본 메트릭으로 충분
API GW 실행 로그	보존기간 무제한 로그 그룹이 자동 생성됨 → 액세스 로그(14일)로 대체
고객 관리형 KMS · Secrets Manager	키당 월 \$1, 시크릿당 월 \$0.4. 관리할 시크릿이 없음(OIDC)
CloudWatch Synthetics · Lambda Insights	실행·함수당 과금. 합성 트래픽은 CI에서 필요할 때만 발생

설계 검토 — 초안의 결함 31건

초안 설계를 그대로 구현하지 않고, 실제 코드로 옮기면서 `validate`, 실제 계정 대상 `plan`, 단위 테스트로 검증했다. 그 과정에서 발견한 결함은 다음과 같다(전체 목록은 `docs/REVIEW.md`).

심각도	초안	문제 → 수정
치명	HTTP API + X-Ray 체인 시각화	X-Ray는 REST API만 지원 → REST 채택
치명	"별칭으로 이전 버전 롤백"	API가 <code>\$LATEST</code> 를 호출해 롤백이 무효 → alias 통합 + 버전 발행
치명	롤백 대상 결정 로직 없음	stable/previous 상태 관리, 플래핑 방지
치명	카나리·알람 기반 검증	저트래픽이라 알람 데이터 없음 → 합성 트래픽
치명	(계정 상태 미확인)	OIDC Provider가 이미 있어 apply 실패 → 재사용 옵션
치명	(구현 중 테스트로 발견)	DynamoDB Decimal이 문자열로 직렬화 → 커스텀 serializer
높음	"초당 10건 스로틀로 비용 방지"	총량 제한이 아님(최악 약 \$90) → 쿼터 + Budgets + 킬스위치
높음	"최소 권한 IAM"	IaC 역할의 권한 상승 경로 → 권한 경계
높음	자동 롤백 + CI/CD	재배포 사고·이중/연쇄 롤백 → 배포 동결, CodeDeploy 위임
높음	로그·메트릭 수집	상세 메트릭 과금, 실행 로그 무제한 보존 → OFF
중간	DynamoDB 쓰로틀 자동복구	On-Demand에서는 시연 불가 → 제거
중간	DynamoDB state 락 테이블	deprecated → S3 네이티브 락

검증

항목	결과
pytest + moto (API 8 · 자동복구 11 · 킬스위치 1)	20 passed
ruff lint / format	통과
terraform fmt / validate (bootstrap, dev)	통과
terraform plan — 실제 AWS 계정, 읽기 전용	bootstrap 11개 · dev 59개 리소스, 오류 없음
실제 배포 · 장애 시연	예정 (예상 비용: 시연 1회 약 \$0.01)

배운 점 · 다음 단계

- "무료"는 설계가 아니다. 무료 티어가 아니라 쿼터처럼 구조적인 상한으로 비용을 보장해야 한다.
- 자동화는 자기 자신과 경합한다. 관리형 롤백과 직접 구현한 롤백, CI가 서로를 되돌리지 않도록 위임·동결 규칙을 명시해야 한다.
- 설계 문서는 코드로 검증해야 한다. 31건 중 다수는 문서만 읽어서는 보이지 않았고, plan과 테스트에서 드러났다.
- 다음 단계: CodeDeploy `BeforeAllowTraffic` 혹은 새 버전 사전 검증, Trivy/checkov IaC 스캔, Slack 알림, 운영 환경 계정 분리.